Application Security Testing — SAST & DAST

Transparency Hub Network Version: 1.0 **Effective Date:** March 5, 2026 **Last Updated:** March 5, 2026 **Owner:** Engineering Team

---

1. Executive Summary

Transparency Hub Network employs both **Static Application Security Testing (SAST)** and **Dynamic Application Security Testing (DAST)** as part of our secure software development lifecycle. These automated security testing measures run continuously across all application repositories to identify vulnerabilities before they reach production.

Testing Type	Tool	Scope	Frequency
SAST	GitHub CodeQL	Source code analysis	Every PR, every push to main, weekly scheduled
DAST	OWASP ZAP	Running application scan	Weekly scheduled, on-demand
SCA	GitHub Dependabot	Dependency vulnerability scanning	Continuous

---

2. Static Application Security Testing (SAST)

2.1 What is SAST?

SAST analyzes source code **before execution** to identify security vulnerabilities, coding flaws, and quality issues. It examines the codebase without running the application, catching issues at the earliest stage of development.

2.2 Tool: GitHub CodeQL

We use **GitHub CodeQL**, an industry-leading semantic code analysis engine, integrated directly into our GitHub CI/CD pipeline.

Why CodeQL:

- Native integration with GitHub (zero additional infrastructure)
- Supports all languages in our stack (Python, JavaScript, TypeScript)
- Uses a powerful query language that models code as data
- Maintained by GitHub Security Lab with continuously updated vulnerability patterns
- Free for open-source and available for private repositories

2.3 What CodeQL Detects

Backend (Python — Azure Functions):

- SQL injection vulnerabilities
- Server-side request forgery (SSRF)
- Code injection and unsafe deserialization
- Path traversal and directory traversal
- Insecure use of cryptographic functions
- Hardcoded credentials and secrets
- Improper input validation

- Insecure temporary file usage
- XML external entity (XXE) processing

Frontend (JavaScript/TypeScript — Next.js):

- Cross-site scripting (XSS)
- Prototype pollution
- Insecure randomness
- Regular expression denial of service (ReDoS)
- Missing or incorrect Content Security Policy
- DOM-based vulnerabilities
- Insecure data flow from user input to sensitive sinks
- Unsafe use of `eval()` and similar constructs
- Client-side URL redirect vulnerabilities

2.4 When SAST Runs

Trigger	Scope	Action on Findings
Pull Request to `main`	Changed files + affected code paths	PR blocked until findings reviewed
Push to `main`	Full codebase scan	Alerts created in Security tab
Weekly schedule (Monday 6:00 AM UTC)	Full codebase scan	Alerts created, team notified

2.5 Configuration

SAST is configured via GitHub Actions workflows in each repository:

Frontend Repository (Staticwebsite-App):

- Language: `javascript-typescript`
- Query suite: `security-and-quality` (extended coverage beyond default security queries)
- Workflow: `.github/workflows/codeql.yml`

Backend Repository (AzureFunction):

- Language: `python`
- Query suite: `security-and-quality`
- Workflow: `.github/workflows/codeql.yml`

2.6 Remediation Process

1. **Alert Created** — CodeQL identifies a potential vulnerability and creates an alert
2. **Triage** — Engineering team reviews the alert within 24 hours
3. **Classification** — Alert marked as: True Positive, False Positive, or Won't Fix (with justification)
4. **Fix** — True positives are fixed on the priority schedule:
    - **Critical/High severity:** Patched within 7 days
    - **Medium severity:** Patched within 30 days
    - **Low severity:** Addressed in next sprint
5. **Verification** — Fix is verified by re-running SAST on the remediation PR

---

3. Dynamic Application Security Testing (DAST)

3.1 What is DAST?

DAST tests the **running application** from the outside — simulating how an attacker would interact with it. Unlike SAST, DAST requires no access to source code and tests the application in its deployed state, including server configurations, authentication flows, and runtime behavior.

3.2 Tool: OWASP ZAP (Zed Attack Proxy)

We use **OWASP ZAP**, the world's most widely used open-source web application security scanner, maintained by the Open Worldwide Application Security Project (OWASP).

Why OWASP ZAP:

- Industry standard for DAST, trusted by organizations worldwide
- Comprehensive coverage of OWASP Top 10 vulnerabilities
- Active open-source community with regular updates
- Docker-based execution for consistent, reproducible scans
- GitHub Actions integration for automated CI/CD scanning
- Free and open source

3.3 What OWASP ZAP Detects

Frontend Scan (Baseline — Passive):

- Missing security headers (X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security, etc.)
- Insecure cookie attributes (missing HttpOnly, Secure, SameSite flags)
- Cross-site scripting (XSS) vulnerabilities
- Cross-site request forgery (CSRF) weaknesses
- Information disclosure (server version headers, error messages, stack traces)
- Mixed content issues (HTTP resources on HTTPS pages)
- Clickjacking vulnerabilities
- Content Security Policy misconfigurations
- Auto-complete enabled on sensitive forms
- Insecure authentication mechanisms

Backend API Scan:

- SQL injection via API parameters
- Authentication and authorization bypass
- Broken access control
- Improper error handling exposing internal details
- API rate limiting issues
- Insecure direct object references (IDOR)
- Mass assignment vulnerabilities
- Missing input validation on API endpoints
- Insecure HTTP methods allowed
- CORS misconfiguration

3.4 Scan Types

Scan Type	Description	Target	Risk Level

Baseline (Passive)	Observes responses without sending attack payloads	Frontend (staging)	Safe — no modification to application
API Scan	Tests API endpoints for common vulnerabilities	Backend API (staging)	Low risk — controlled payloads

Important: Active scans with aggressive payloads are only run against the **staging environment** (`testapp.transparencyhubnetwork.ai`), never against production.

3.5 When DAST Runs

Trigger	Scan Type	Target
Weekly schedule — Monday 7:00 AM UTC	Baseline scan	Frontend staging
Weekly schedule — Monday 8:00 AM UTC	API scan	Backend API staging
Manual trigger (on-demand)	Baseline or API scan	Staging environment

3.6 Reporting

DAST results are delivered through multiple channels:

1. **GitHub Issues** — ZAP automatically creates or updates a GitHub issue titled "ZAP DAST Scan Report" with categorized findings
2. **HTML Reports** — Detailed reports uploaded as GitHub Actions artifacts (retained 30 days)
3. **Alert Classification** — Each finding includes:
    - Risk level (High, Medium, Low, Informational)
    - Confidence level
    - Affected URL/endpoint
    - Description and recommended fix
    - OWASP Top 10 / CWE reference

3.7 Tuning and False Positives

We maintain a rules configuration file (`.zap/rules.tsv`) in each repository to:

- Suppress known false positives
- Adjust alert thresholds for environment-specific configurations
- Ignore findings that are mitigated by infrastructure controls (e.g., Azure Static Web Apps handles CSP headers)

---

4. Software Composition Analysis (SCA)

In addition to SAST and DAST, we employ **Software Composition Analysis** to monitor third-party dependencies.

4.1 Tool: GitHub Dependabot

- **Continuous monitoring** of all dependencies for known vulnerabilities
- **Automated pull requests** to update vulnerable packages
- Covers both direct and transitive dependencies
- Alerts categorized by severity (Critical, High, Medium, Low)

4.2 Dependency Audit

- `npm audit` (frontend) and `pip audit` (backend) available for on-demand scanning
- npm overrides used to patch transitive dependency vulnerabilities when upstream fixes are unavailable

---

5. Security Testing Coverage Matrix

OWASP Top 10 (2021)	SAST (CodeQL)	DAST (ZAP)	SCA (Dependabot)
A01: Broken Access Control	✅	✅	—
A02: Cryptographic Failures	✅	✅	—
A03: Injection	✅	✅	—
A04: Insecure Design	Partial	—	—
A05: Security Misconfiguration	✅	✅	—
A06: Vulnerable Components	—	—	✅
A07: Auth Failures	✅	✅	—
A08: Software/Data Integrity	✅	—	✅
A09: Logging/Monitoring Failures	Partial	✅	—
A10: Server-Side Request Forgery	✅	✅	—

---

6. Repositories and Workflow Locations

Repository	SAST Workflow	DAST Workflow	SCA
Staticwebsite-App (Frontend)	`.github/workflows/codeql.yml`	`.github/workflows/zap-scan.yml`	Dependabot (enabled)
AzureFunction (Backend)	`.github/workflows/codeql.yml`	`.github/workflows/zap-scan.yml`	Dependabot (enabled)

---

7. Compliance and Standards

Our security testing practices align with:

- **OWASP Application Security Verification Standard (ASVS)** — Testing coverage maps to ASVS Level 2
- **OWASP Top 10 (2021)** — All categories addressed through SAST, DAST, or SCA
- **Nigeria Data Protection Regulation (NDPR)** — Security testing supports compliance with data protection requirements
- **ISO 27001** — Aligned with Annex A.14 (System Acquisition, Development, and Maintenance)

---

8. Continuous Improvement

- Security testing rules and configurations are reviewed quarterly

- New vulnerability patterns are added as they emerge
- False positive rates are monitored and tuning rules updated accordingly
- Coverage gaps identified through incident reviews are addressed with additional custom queries
- Annual review of tooling to evaluate newer/better alternatives

---

9. Document Control

Version	Date	Author	Changes
1.0	2026-03-05	Engineering Team	Initial version

This document is reviewed and updated quarterly, or immediately following a significant security incident or tooling change.

For questions about this document, contact: support@transparencyhubnetwork.ai